Foundation Health

Assignment

Technical Assessment: MP3 File Analysis App

Objective

The aim of this task is to create an API endpoint according to the specification below that accepts an MP3 file and responds with the number of frames in the file. A sample file is provided for candidates to test their solution against.

This assessment will test the candidate's ability to:

- Learn & apply new knowledge in a likely unfamiliar context
- Demonstrate technical competency in Typescript
- Show personal coding styles & preferences

API Specification

- The application **must** host an endpoint at `/file-upload` that accepts an MP3 file upload via the `POST` method.
- The endpoint **must** successfully count the number of frames for **MPEG Version 1 Audio Layer 3** files (This is almost universally the format for `.mp3` files, including the provided sample).
- The endpoint **must** provide a JSON response (with the correct response headers) in the following format:

⚠ `Handling of other MPEG file formats should be considered out of scope for this exercise & candidates are encouraged not to spend time on this.`

- The endpoint **must** provide a JSON response (with the correct response headers) in the following format:

```
{
  "frameCount": <number>
}
```

Note: <number> should be the correct number of frames in the MP3 file

Requirements

The candidate:

- **Must** use TypeScript to complete the exercise.
- **Must** parse the MP3 file to logically count the number of frames in the file.
- **Must not** use an NPM package to parse the MP3 frame data directly. NPM packages may be used for other things such as generic utilities or a http framework.
- **Must** respond to the client with the correct number of frames found in the file, in the format specified above.

Evaluation Criteria

The candidate's solution will be evaluated based on the following criteria:

Correctness

- Does the solution meet the requirements?
- Does the solution correctly determine the number of frames in the MP3 file correctly?
- Does the solution handle errors appropriately?

Code Quality

- Is the code well-organised, readable, and maintainable?
- Does the code follow TypeScript good practices?

- Does the code utilise TypeScript's features effectively?
- Does the solution include standardised tooling for formatting, linting, testing etc?

Error Handling

- Is error handling implemented appropriately?
- Does the solution handle errors gracefully?
- Does the solution provide useful error messages to the user?

Scalability

- Is the solution scalable and able to handle large files?
- Is the solution optimised for performance?

Approach

- Has the candidate used Git effectively?
- Is there evidence of a structured approach to the solution?

Tips

- The candidate may wish to use a tool such as `mediainfo` to verify their results.
- Candidates are reminded to test final solutions before submission.
- Priority should be given to producing a working solution in the allotted time - *Done is better than perfect*. Brief comments may be left to explain optimisations / improvements that would be implemented with more time & there will be an opportunity to expand on this at interview.
- Candidates may use AI assistance to complete the exercise but the submission will still be taken as the candidates own work and therefore it should be representative of something they would write themselves and they will be expected to understand and explain the solution.

Submission

The candidate should provide the solution as a **git** repository & should include clear instructions for running the application & an example of how to test the application.

Timeframe

The candidate should aim to complete the assessment within **5 days** of receiving the task.

Good luck! We look forward to seeing your solution.

Submission

Files

Upload File(s)

or drag and drop here

Notes

Write here...

Submit Assignment